LIÈGE université
Sciences Appliquées

AI-DJ: Spotify Hit Predictor

Presented by : Antoine Paulis
Charlotte Michon
Mohamed-Khalil Ankri

LIÈGE université
Sciences Appliquées

Presentation Outline

I. Project Overview

II. Backend and Frontend

III. Pipeline and Deployment

IV. Demo

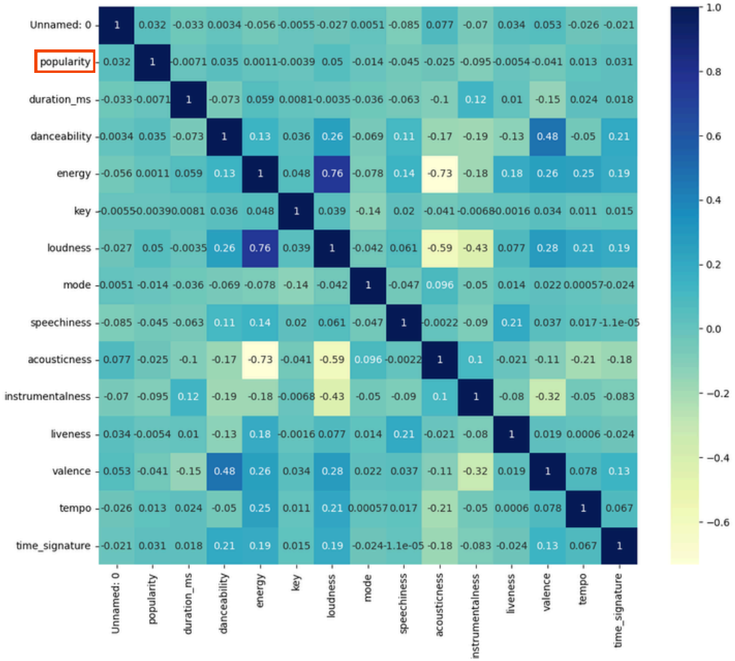
1. Project Overview

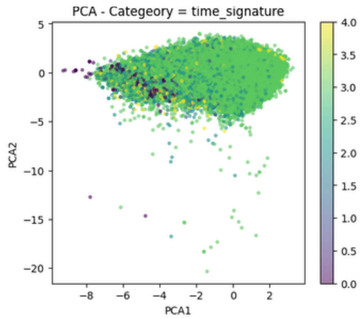
Project Definition

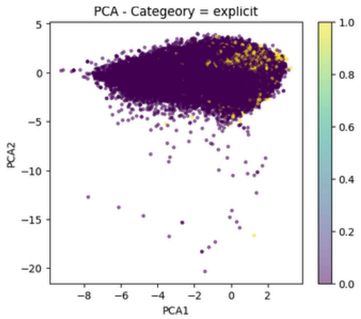
- **Project proposal :** Model to predict if a song will be a “**hit**” or a “**flop**”, by training on a Spotify dataset of different genres of music.
- **Dataset :** Spotify dataset (*kaggle*) composed of **21** variables and **114,000** tracks from 125 genres of music.
  - **Metadata & identifiers :** track_id, artists, album_name, track_name and track_genre.
  - **Audio features :** loudness, acousticness, duration_ms, etc (different scales).
  - **Target variable:** Popularity (1-100) → Hit (0-1) with threshold at 65.
    - Total number of streams and how recent those streams are.
- **Problem :** Binary classification.

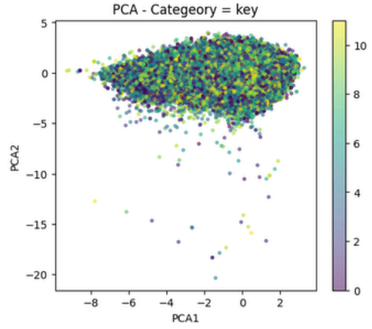
EDA results

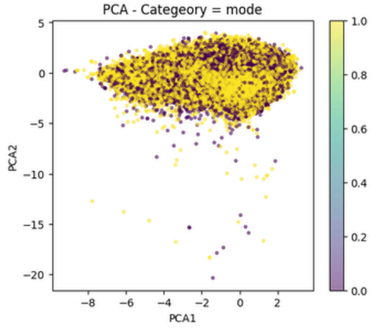
- **Class imbalance :** 8.31% of Hits.
- **Correlation:**
  - Popularity → No strong correlation with any features.
  - Overall absolute correlation is low.
- **PCA :**
  - No clear cluster.
  - Information spread across variables.





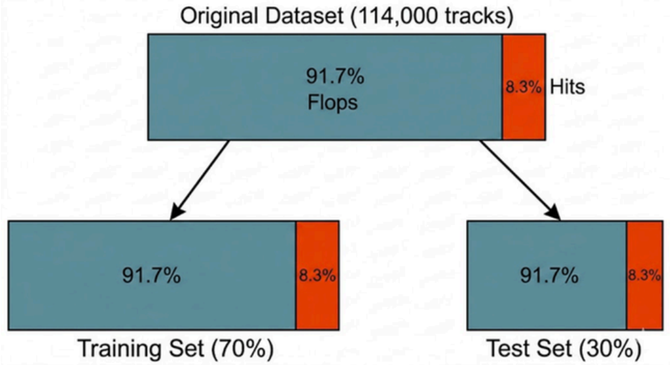


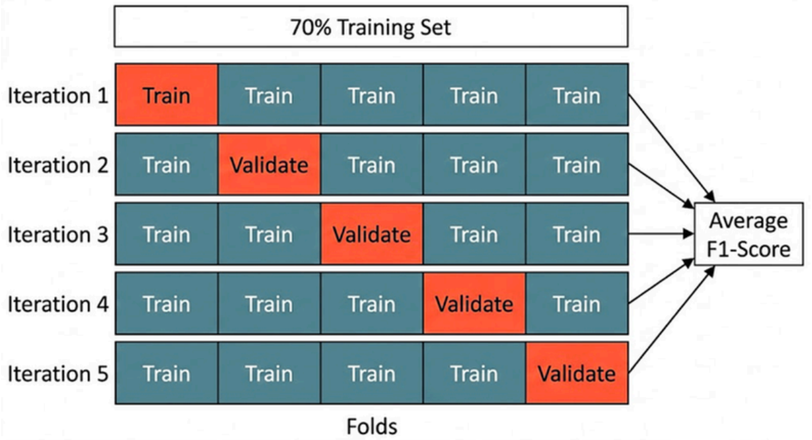




ML Model

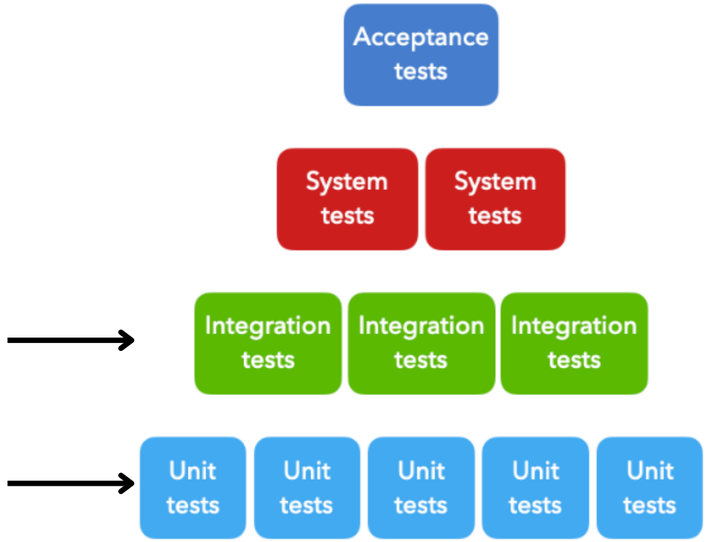
- **Experiment Setup :**
  - **Data Split :** 70/30 Stratified Split .
  - **Validation :** Stratified K-Fold Cross-Validation (with shuffle).
  - **Decision metrics :** F1 score.
  - **Algorithms Tested :**
    - KNN : F1 = 0.67.
    - **Random Forest : F1 = 0.65 →** class imbalance management, faster and less memory.
    - Gradient Boosting : F1 = 0.39.





MLOps Foundation

- **Google cloud :** BigQuerry : tabular data + better querrying and scalability.
- **CI/CD - Github actions :**
  - **Unit tests :** tests of metricsEval(yValFold, yPred).
  - **Integration Test :** tests of importBigQuerry() and trainingPreprocess().
  - **Code format checks :** ruff.
  - **Pre-commit hooks**



2. Backend & Frontend

Backend & Frontend:
Api, UI and Application Layer

Backend: FastAPI Service

REpresentation State Transfer API:

- **Manages the full prediction pipeline**
- **Model not loaded locally, every prediction calls our Vertex AI endpoint**
- **All errors return standard FastAPI HTTP exceptions with an error message**

Technologies:

- **FastAPI: Pydantic, Swagger and uvicorn**
- **Container: Docker (py 3.12 slim) + uv**
- **Google Cloud Run**
- **scikit-learn for ML prediction**

Google Cloud Run deployment:

Host = europe-west4 and west1

5-steps per /predict

1. **Audio features: Musicae RapidAPI (get)**
2. **Track metadata; Musicae (get)**
3. **ML inference: uses Vertex AI endpoint**
4. **Explanation: local run (no API call), rule-based**
5. **Recommendations of similar tracks: Musicae (get)**

Endpoints

main.py: src/api/

- **GET /health: simple check, {"status":"ok","mode":"vertex-ai"}**
- **POST /predict: Spotify track URL or ID, runs the full pipeline**
- **POST /predict_manual: user enters features**
- **GET /past_predictions: history**

Backend & Frontend:
Api, UI and Application Layer

Frontend: Streamlit UI = SIGNAL/STUDIO

Home page: 3 tabs

- **Analyze: paste a Spotify track URL,**
  - **returns prediction hit or not + explanation = Track Analysis,**
  - **displays Key drivers: 5 most impactful features (accousticness, energy, loudness, valence, instrumentaless)**
  - **provides Feature radar: visualization**
- **Manual Input: 9 sliders + 4 parameter selectors for custom features predictions**
- **History: displays past predictions fetched**

EDA page: dataset insights

- **Dataset Description: summary statistics, hit/non-hit balance, features distributions**
- **Correlation Matrix, Pair Plot, PCA 2D and PCA 3D visualizations**

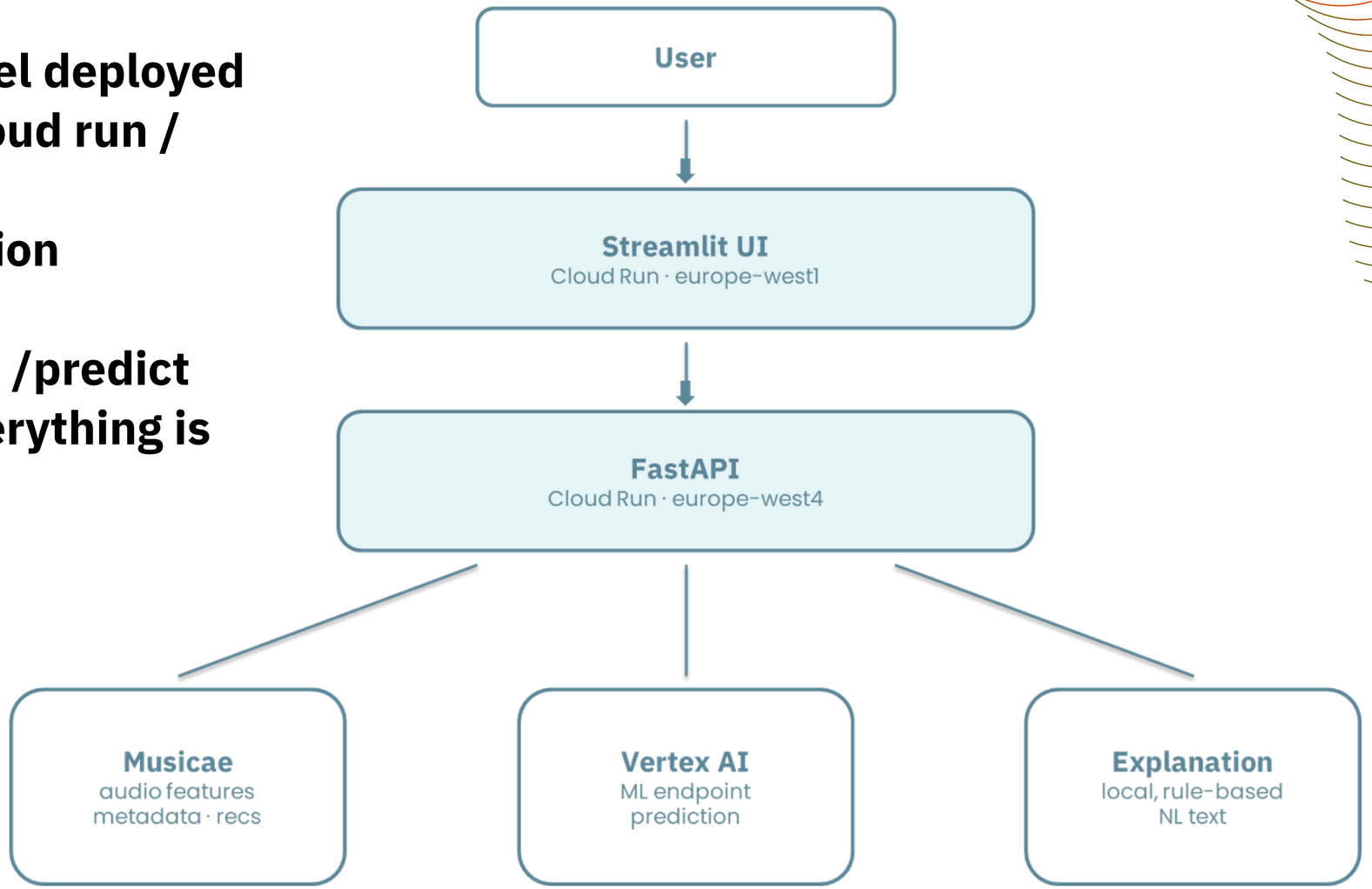
Technologies:

- **Streamlit**
- **Plotly: radar charts and 3D PCA interaction**
- **Matplotlib**
- **custom CSS (for the dark theme)**

Backend & Frontend:
Api, UI and Application Layer

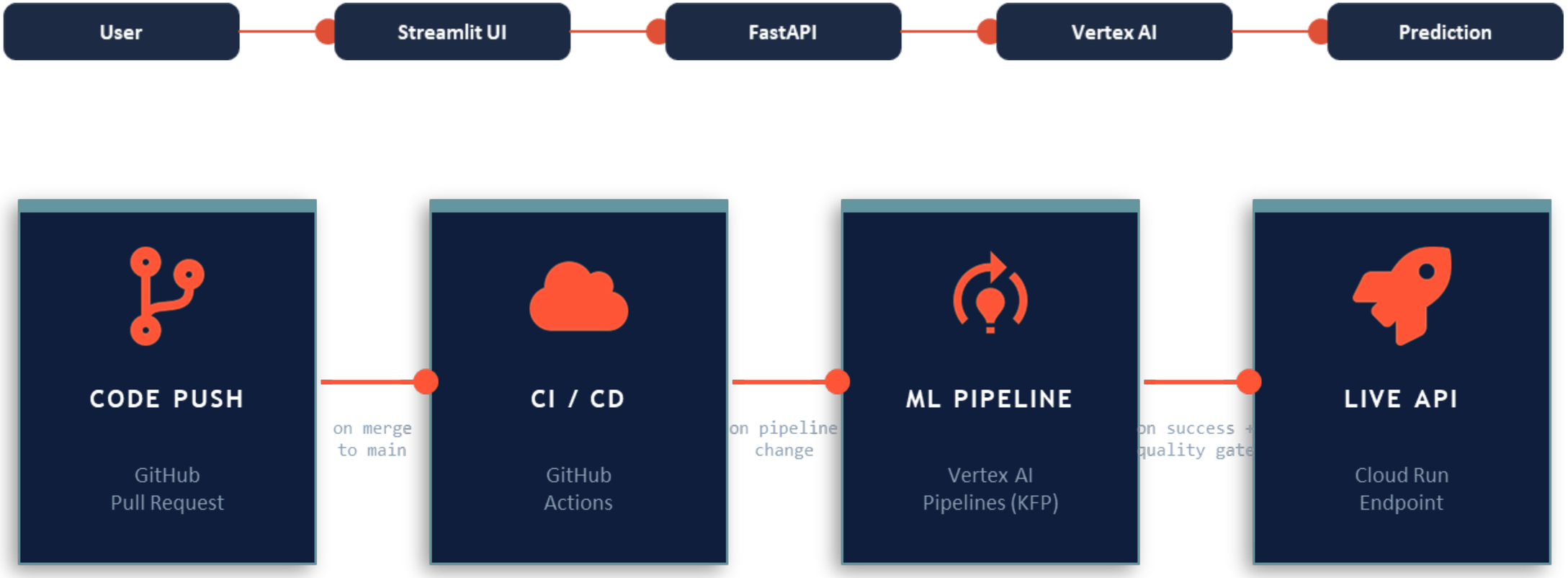
End-to-End Data Flow

- **Frontend, backend and model deployed independently on Google Cloud run / Vertex API**
- **Authentification and validation centralised in the API**
- **Stateless request flow: each /predict triggers the full pipeline, everything is kept in history**



3. Pipeline & Deployment

System Architecture



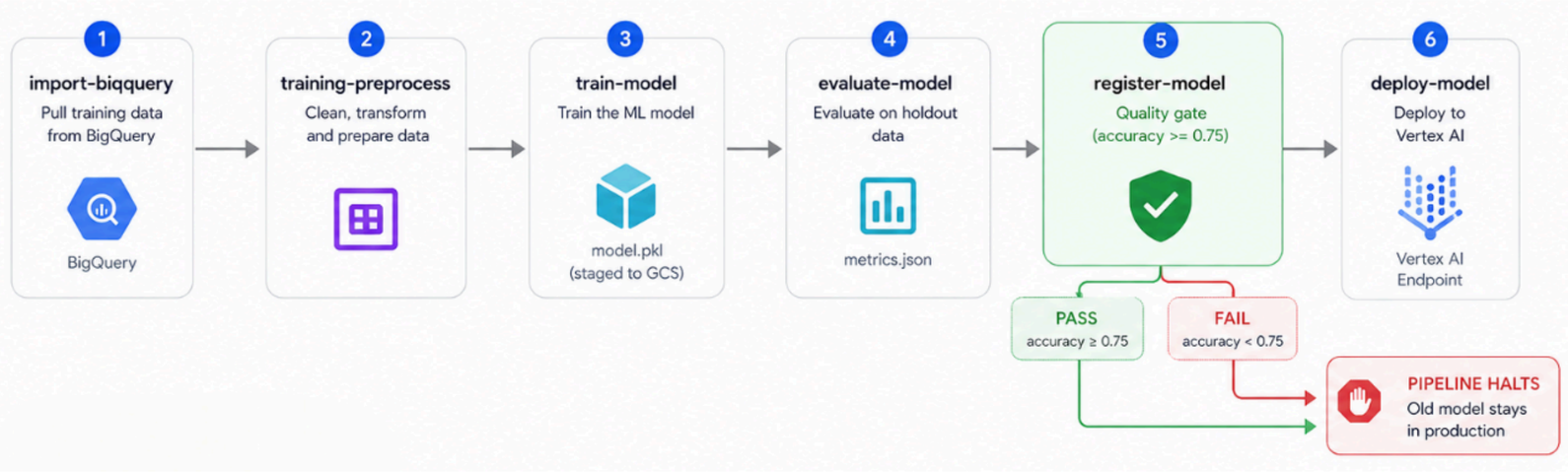
4
GitHub Workflows

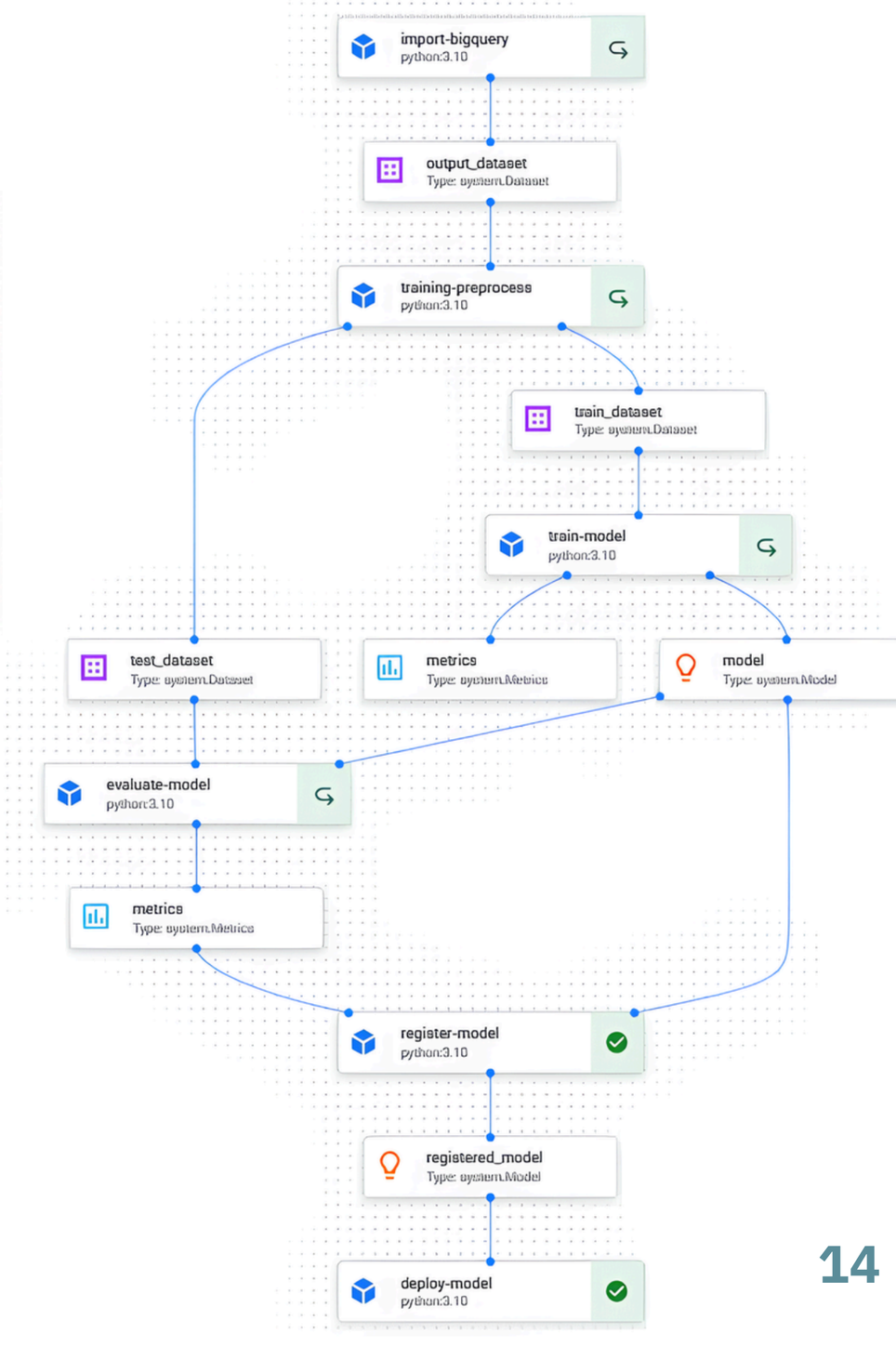
6
KFP Pipeline Stages

75%
Accuracy Threshold

2
Cloud Run Services

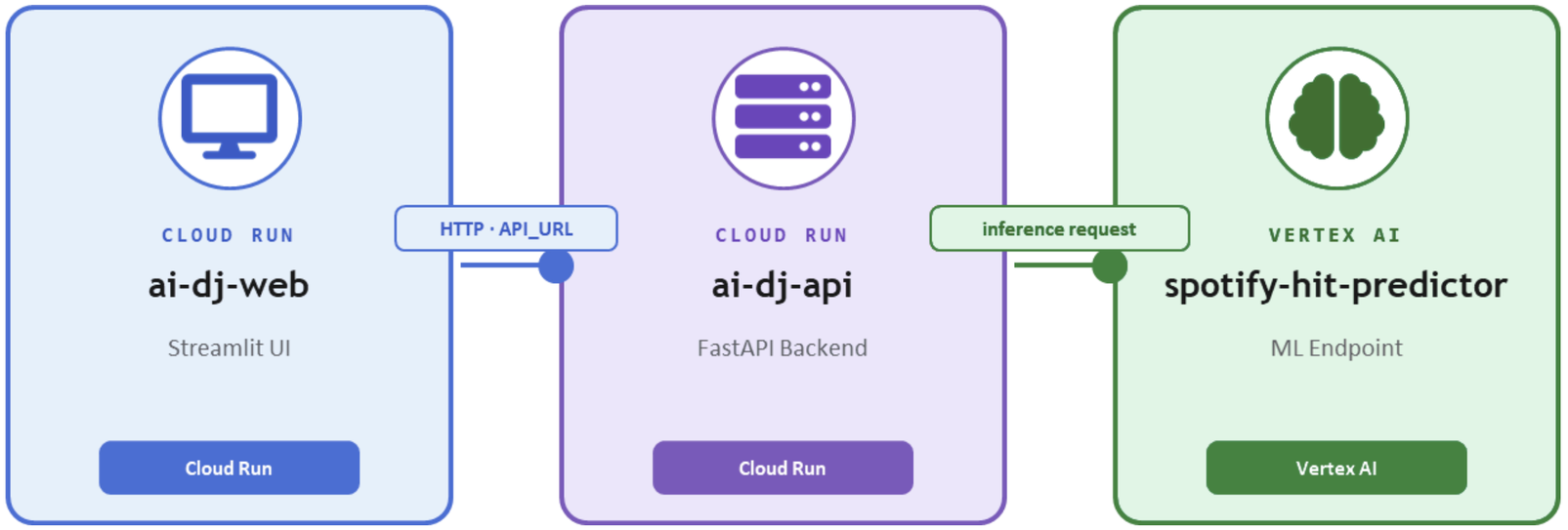
The KFP Pipeline





- **6 sequential KFP components running on Vertex AI Pipelines**
- **Quality gate at stage 5 : accuracy ≥ 0.75 required, otherwise pipeline halts and current model stays live**
- **Model artifact staged to GCS before registration to meet Vertex AI's serving requirements**
- **Idempotent deployment : existing endpoint reused, no duplicates created**

Deployed Infrastructure



- **UI → API only ： the frontend never talks to Vertex AI directly.**
- **Stateless API ：no database; predictions are handled in-memory.**
- **Fully decoupled services ： frontend and backend deploy independently.**
- **Fully managed ： no server provisioning; Cloud Run and Vertex AI scale automatically.**
- **Model never exposed : all requests go through the API, which handles validation and formatting.**

CI/CD & the 4 Workflows

Path-based Triggers (Cloud Build)

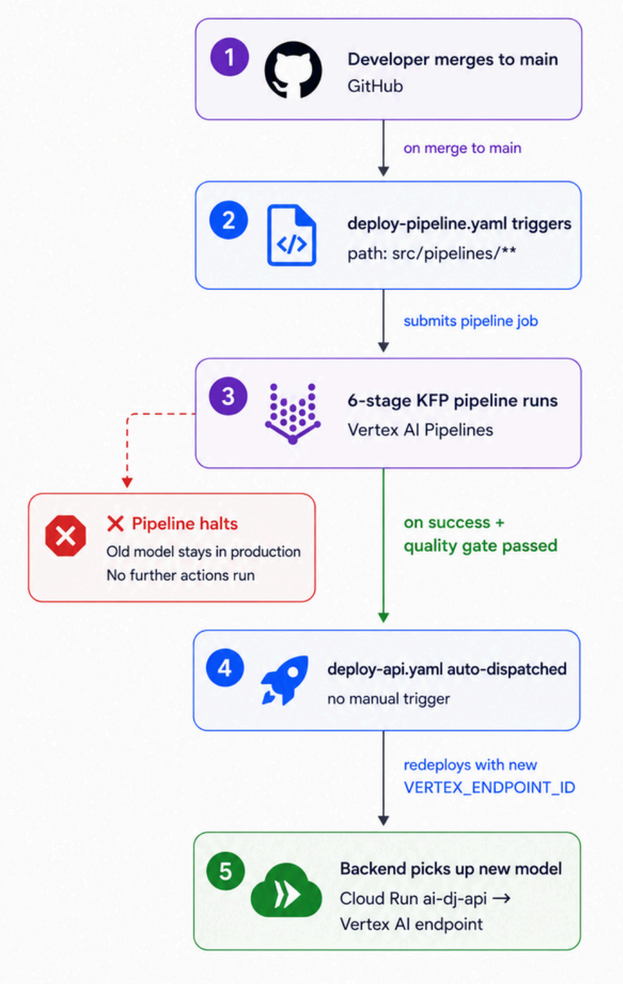
Only the workflows affected by changes run

Workflow	Triggered By changes in ..
ci.yaml	Any file
deploy-api.yaml	src/api/**, Dockerfile.backend
deploy-web.yaml	src/ui/**, Dockerfile
deploy-pipeline.yaml	src/pipelines/**, pipeline.py

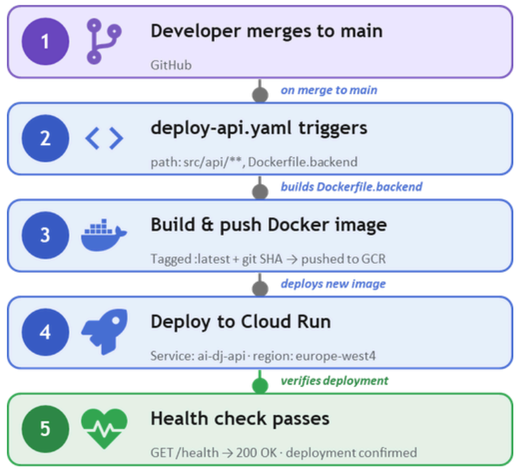
Path-based Triggers (Cloud Build)

ci.yaml	Lint (flake8) + smoke tests (pytest)
deploy-api.yaml	Build & deploy backend to Cloud Run (ai-dj-api)
deploy-web.yaml	Build & deploy frontend to Cloud Run (ai-dj-web)
deploy-pipeline.yaml	Compile KFP pipeline & submit to Vertex AI

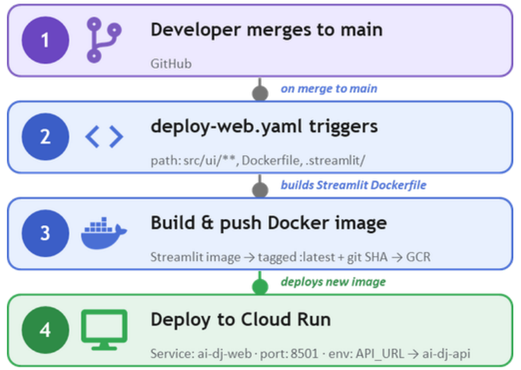
The Chain Reaction



The Chain Reaction : API

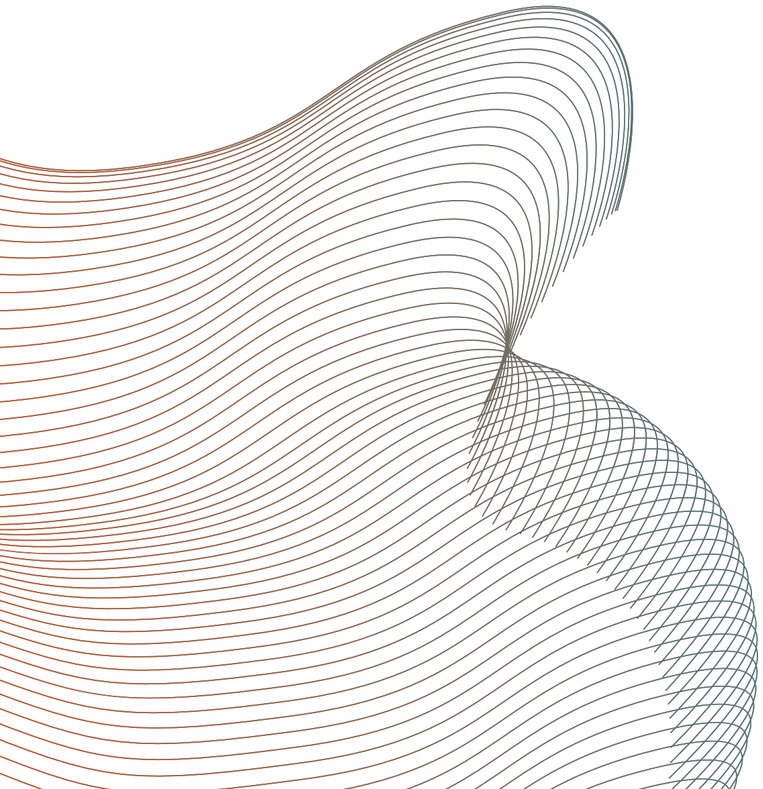


The Chain Reaction : UI



Demo

LINK https://ai-dj-ui-343602206157.europe-west1.run.app/



Q & A